CTU training solutions | Private Tertiary Education

CAPSTONE PROJECT

CHAPTER 3 - REQUIREMENTS & METHODOLOGY

Student / Group Template

Project Title	Campus Help Desk Ticketing System: A Database-Driven Application for IT Support Request Management
Group Members	Freek Schaefer-Gerber and Wian Sandenbergh
Module	SPM622 / MDB622
Lecturer / Supervisor	Mr. Dunstan
Week / Stage	Requirements & Methodology finalised
Submission Date	29 October 2026

CTU Training Solutions (PTY) Ltd is fully accredited by the MICT SETA (ACC/2006/07/186) through the Quality Council for Trades and Occupations (QCTO). CTU Training Solutions is registered with the Department of Higher Education and Training as a Private College (2018/FE07/004) and provisionally registered as a private higher education institution (No 2014/HE07/004) Co. Reg No:2005/038944/07

3.1 Introduction

Introduce the approach used to gather requirements and develop the system.

This chapter specifies the proposed customer support ticketing system and the method planned for its development. Requirements are based on scenario analysis and examination of common helpdesk workflows; they are preliminary until reviewed with the lecturer or representative users. The project focuses on customer ticket submission, agent handling, administrative oversight and reporting.

3.2 Stakeholders and Users

Identify primary and secondary stakeholders and their needs.

Customers report issues, track progress and respond to support agents. Support agents review assigned tickets, communicate with customers and resolve issues. Administrators manage assignments, categories, roles and summary reports. The lecturer and project evaluators assess functionality, documentation, evidence and usability. The development team comprises Freek Schaefer-Gerber and Wian Sandenbergh.

3.3 Functional Requirements

List what the system must do. Give each requirement an ID, description and priority.

Each proposed functional requirement has a unique identifier and acceptance condition. Must items form the minimum viable product (MVP); Should items are targeted after core flows work.

ID	Functional requirement	Priority
FR-01	Users can register and sign in securely	Must
FR-02	Customer creates ticket with subject, category, description and priority	Must
FR-03	Customer views own tickets and status history	Must
FR-04	Agent views and updates assigned tickets	Must
FR-05	Administrator assigns tickets to agents	Must
FR-06	Customer and agent exchange ticket comments	Must
FR-07	Admin manages user roles and ticket categories	Should
FR-08	Dashboard displays ticket	Should

	counts by status and priority	
FR-09	Users search/filter tickets by status and category	Should
FR-10	System records creation and status-change timestamps	Must
FR-11	Customer may attach an image to a ticket	Could
FR-12	Agent can resolve ticket and customer can confirm closure	Should

3.4 Non-Functional Requirements

Document usability, performance, security, reliability, maintainability and other quality requirements.

Quality requirements define how the application should behave. The proposed targets below are intended for testing, not claims of measured performance.

ID	Quality attribute	Proposed acceptance target
NFR-01	Security	Passwords hashed; authorization enforced server-side for each ticket operation
NFR-02	Privacy	Customers can only access their own tickets; no real customer data in demonstrations
NFR-03	Usability	Core ticket submission can be completed on mobile and desktop without horizontal scrolling
NFR-04	Performance	Under local test conditions, 95% of typical ticket-list requests complete within 2 seconds with 100 sample tickets
NFR-05	Reliability	Invalid inputs yield controlled errors without corrupting existing tickets
NFR-06	Maintainability	Separate UI, API/business

		logic and persistence layers; version control all code
NFR-07	Data integrity	Primary/foreign keys, required fields and permitted status values validated
NFR-08	Accessibility	Forms have visible labels, keyboard navigation and readable contrast

3.5 Requirements Gathering

Explain how requirements were identified, such as interviews, observation, questionnaires, document analysis or scenario analysis.

Requirements were drafted using scenario analysis of a small organization receiving customer complaints through email, calls and informal messages, along with feature comparison of established ticketing approaches. No interviews, questionnaires or field observations are claimed at this stage. Before implementation is finalized, the team should validate role permissions, required ticket fields and workflows through a short stakeholder review and record feedback.

3.6 Development Methodology

Describe the chosen SDLC/Agile approach and justify why it fits this project.

A lightweight Agile iterative approach is proposed because the deadline is 29 October 2026 and the team consists of two developers. Iteration 1: finalize scope, database schema and authentication. Iteration 2: implement ticket creation, lists and authorization. Iteration 3: assignment, comments, status updates and dashboards. Final iteration: execute test cases, resolve critical defects, capture Git evidence, and assemble dissertation and demonstration. A shared Git repository, issue board and short daily check-ins will support coordination. Tasks should be assigned by agreement, with both members reviewing integration and testing.

3.7 Use Cases / User Stories

Document key interactions between users and the system.

US-01: As a customer, I want to create a support ticket so that my issue is recorded. US-02: As a customer, I want to view my tickets so that I can track progress. US-03: As an agent, I want to view assigned tickets so that I can respond promptly. US-04: As an administrator, I want to assign tickets so that work is distributed. US-05: As an agent, I want to update ticket status and comment so that customers are informed. US-06: As an administrator, I want to view summary counts so that I can monitor workload.

3.8 System Architecture

Describe the proposed architecture and major components.

The implemented system uses a three-layer web architecture: (1) a browser-based frontend rendered using Jinja2 HTML templates; (2) a backend application layer built with Python and the Flask framework, enforcing validation, authorization, and business rules; and (3) a relational SQLite database storing users, tickets, and comments. All protected routes check the authenticated user's role and ownership server-side.

3.9 Ethical, Privacy and Security Considerations

Explain responsible handling of users, data, consent, privacy and security.

The demonstration should use fictional accounts and tickets. Collect only necessary fields, avoid storing passwords in plain text, enforce role-based access, validate input and restrict attachment type and size if attachments are implemented. Keep secrets out of the Git repository and use environment variables. Obtain consent before collecting any real user feedback and follow applicable institutional policies and South African privacy requirements, including POPIA where relevant. Do not claim ethical clearance unless granted.

3.10 Chapter Summary

Summarise how the requirements and methodology guide development.

This chapter defines a role-based customer support system, a prioritized requirements baseline, and an iterative implementation plan. The next chapter should translate these requirements into architecture diagrams, database entities, interfaces and demonstrable code. Requirement IDs will also be reused in the test plan and traceability matrix.

CTU Training Solutions (PTY) Ltd is fully accredited by the MICT SETA (ACC/2006/07/186) through the Quality Council for Trades and Occupations (QCTO). CTU Training Solutions is registered with the Department of Higher Education and Training as a Private College (2018/FE07/004) and provisionally registered as a private higher education institution (No 2014/HE07/004) Co. Reg No:2005/038944/07